

Exchange Health Check — Guida utente

Monitoraggio periodico per ambienti Exchange on-premises multi-server, con alerting via e-mail. Controlla servizi, spazio disco, stato del DAG e delle copie di database, code di trasporto, certificati e Managed Availability. Lavora in **sola lettura**: nessun cmdlet di scrittura verso Exchange.

Repository: <https://github.com/diepo/ExchangeHealthCheck>

1. Cosa controlla

Area	Dettaglio
Connettività	Server raggiungibile via WinRM/CIM
Sistema operativo	Uptime, reboot in sospeso, memoria, CPU
Disco	Spazio libero su ogni volume, incluse le mount point senza lettera
Dischi fisici	Stato di salute di ogni disco fisico (Get-PhysicalDisk): Healthy/Warning/Unhealthy. Su VM i dischi virtuali dell'hypervisor ("VMware Virtual disk") sono esclusi: non riflettono lo storage fisico reale
Servizi	Servizi Exchange fermi che dovrebbero essere in esecuzione
Componenti	Server lasciato in maintenance mode dopo un patching
Managed Availability	Health set non sani, con i monitor coinvolti in dettaglio
DAG	Membri fermi, witness, nodi del cluster
Copie di database	Stato delle copie, copy/replay queue, content index
Replica	Test-ReplicationHealth
Database	Database non montati, copia attiva fuori preferenza 1
Code di trasporto	Code accumulate, submission, poison, retry, back pressure
Certificati	Scadenza, stato non valido, duplicati che confondono i connector

2. Requisiti

- Windows PowerShell 5.1 o superiore (va bene anche PowerShell 7.x)

- Da eseguire preferibilmente **su un server Exchange**, oppure da una macchina con `Organization.ConnectTo` configurato per aprire una remote session
- Account con ruolo RBAC **View-Only Organization Management** e amministratore locale sui server (serve per il fan-out WinRM)
- WinRM abilitato sui server target

3. Installazione

1. Copia la cartella del progetto su un server di gestione o su un server Exchange.
2. Crea la tua configurazione a partire dal template:

```
powershell Copy-Item .\ExchangeHealthCheck.config.example.json  
.\ExchangeHealthCheck.config.json
```

Il file `ExchangeHealthCheck.config.json` contiene dati del tuo ambiente (nomi server, indirizzi mail) e non va mai condiviso o versionato.

3. Apri il file e imposta almeno le chiavi seguenti:

- `Organization.ViewEntireForest`: `false` se la tua organizzazione Exchange vive in un solo dominio (caso più comune)
- `Servers.Include`: i server da controllare (o `["*"]` per tutti)
- `Mail.SmtptServers`, `Mail.From`, `Mail.To`

4. Primo test, passo per passo

4.1 — Un solo check, senza inviare nulla

```
.\Invoke-ExchangeHealthCheck.ps1 -Server NOME-SERVER -Check Disk -NoMail -Verbose
```

Se questo funziona, sono confermati i tre presupposti di base: i cmdlet Exchange rispondono, WinRM verso quel server funziona, i dati tornano.

4.2 — Tutti i check, ancora senza inviare nulla

```
.\Invoke-ExchangeHealthCheck.ps1 -NoMail -Verbose
```

Con `-NoMail` lo stato degli alert non viene mai marcato come notificato: puoi ripetere il test tutte le volte che vuoi senza perdere alert veri quando attiverai davvero la mail.

4.3 — Leggi il report

Ogni esecuzione produce un CSV in `Reports\`:

```
$ultimo = Get-ChildItem .\Reports\*.csv |  
Sort-Object LastWriteTime -Descending |  
Select-Object -First 1  
  
Import-Csv $ultimo.FullName |  
Where-Object Severity -ne 'OK' |  
Format-Table Severity, Server, Category, Message -AutoSize
```

Usa questo passaggio per tarare le soglie e popolare `Ignore.*` prima di attivare le mail: il primo giro con la mail attiva invia tutte le anomalie già presenti in quel momento, in un unico messaggio.

4.4 — Prova solo l'invio mail

```
.\Invoke-ExchangeHealthCheck.ps1 -TestMail
```

Non esegue alcun controllo: manda solo un messaggio di prova e mostra i parametri SMTP effettivamente usati. Se fallisce, il log riporta la causa reale (host non risolvibile, connessione rifiutata, destinatario non accettato, autenticazione richiesta).

4.5 — Primo giro vero

```
.\Invoke-ExchangeHealthCheck.ps1
```

5. Come funziona l>alerting

Lo stato di ogni anomalia è persistito in `State\alert-state.json` con chiave `Categoria|Server|Oggetto`. Ad ogni esecuzione:

- **anomalia nuova** → mail immediata
- **anomalia peggiorata** (es. Warning → Critical) → mail immediata
- **anomalia già nota, non peggiorata** → nessuna mail finché non scade `Alerting.CooldownMinutes` (default 120 minuti), poi un promemoria
- **anomalia rientrata** → mail di rientro con la durata del disservizio
- **nessuna anomalia da giorni** → un heartbeat ogni `Alerting.HeartbeatHours` (default 24), per sapere che il monitor è vivo

Se l'invio della mail fallisce, l'anomalia **non** viene marcata come notificata: il giro successivo riprova, non resta in silenzio per il cooldown.

Colpo d'occhio sull'ambiente. La mail (e la console, durante un giro a secco) apre con un riepilogo **per categoria** (Disk, Service, Queue, Certificate, ...): quante cose sono Critical/Warning in ciascuna, prima ancora del dettaglio server per server. Risponde a "cosa non va nell'infrastruttura" senza dover aprire ogni tabella. Il numero conta i **problemi distinti**, non un'occorrenza per server: se lo stesso health set o lo stesso certificato è segnalato identico su 5 server dello stesso DAG (spesso è davvero la stessa causa), conta 1, non 5. Due dischi pieni su server diversi, con GB liberi realmente diversi, restano invece due voci — non sono la stessa cosa.

Le tabelle di dettaglio sotto ("Nuove anomalie", "Anomalie peggiorate", "Anomalie ancora aperte", "Low Issue") seguono lo stesso principio: una riga per **evento** (stessa categoria, stesso oggetto, stessa severità), non una riga per server. Se coinvolge più server, li trovi tutti elencati nella colonna "Server" in fondo alla riga — per un disco, ciascun server è seguito tra parentesi dal proprio spazio libero (es. `PC30 (4.2 GB / 3.5%)`), perché in quel caso il numero cambia davvero da un server all'altro.

Riepiloghi per cluster e per database. Sullo stesso principio, la mail mostra anche una vista aggregata dello **stato del cluster per DAG** (nodi totali, quanti attivi, quali fermi e il loro stato) e delle **copie database per server** (quante sane, quante no, con il dettaglio). Raccolte sempre, non solo quando c'è un problema — così vedi subito se un nodo è `Down` o una copia è `Failed/Seeding` senza dover interrogare manualmente ogni server.

Riepilogo per database. Oltre alla vista per server, una vista dedicata per **database**: per ciascuno, su quale server è la copia attiva (`Active`), e per ogni altra copia il suo stato grezzo di Exchange (`Healthy` = in sync, `Seeding/Suspended/Failed/...` = no), con l'indicazione del content index solo quando non è nello stato normale. Risponde direttamente a "questo database, chi lo serve adesso, e le altre copie sono allineate?" senza dover incrociare a mano le righe della vista per server. Se nessuna copia risulta attiva, il database compare comunque,

marcato Critical. Ordinata per nome database (numerico, non alfabetico puro: DB2 prima di DB10), non per gravità — è un inventario che si scorre sempre nello stesso ordine, i problemi restano comunque visibili dal colore della riga. Per ogni copia non attiva mostra sempre la sua replay queue (RQ:), la copy queue solo se diversa da zero (CQ:) e un eventuale commento di sospensione — non solo quando superano la soglia, per non dover controllare a mano `Get-MailboxDatabaseCopyStatus` ogni volta che una riga sembra sospetta.

Riepilogo dischi fisici. Una riga per ogni disco fisico di ogni server (`Get-PhysicalDisk`): modello, tipo, dimensione, stato di salute e stato operativo. Nato da un caso reale: un database restava indietro con la replay queue bloccata a migliaia di log mentre gli altri sullo stesso server stavano bene — la causa era un disco fisico degradato, visibile solo aprendo Server Manager, non da alcun sintomo del database stesso. Su uno storage **JBOD** (un disco dedicato per database, niente RAID sui volumi dati — la ridondanza la dà già il DAG) un disco che degrada colpisce esattamente un database, lasciando sani tutti gli altri: questa vista lo rende visibile subito, senza dover indovinare quale sia la causa da un sintomo indiretto. In console i dischi non sani vengono mostrati in cima (qui l'obiettivo è vedere subito i pochi problemi su un server che può avere decine di dischi, non scorrere un inventario fisso). **In mail** la sezione mostra solo i dischi non sani, e sparisce del tutto se sono tutti Healthy — l'inventario completo (utile per un'ispezione manuale) resta solo in console/report, non ha senso ripeterlo ad ogni mail su un parco di decine di dischi.

Due mail distinte. Oltre alla mail di riepilogo (sempre completa: contatori, categorie, server, dettaglio), quando in un giro compare qualcosa di davvero nuovo o peggiorato ne parte una seconda, minimale, con oggetto **"WARNING FOUND"** e solo le righe rilevanti — pensata per essere letta in pochi secondi. Scatta solo su:

- un **nuovo Critical** in qualunque categoria (o un Warning che diventa Critical);
- una **cod**a che raggiunge `Thresholds.QueueSubjectThreshold` (default 200 messaggi), a prescindere dalla sua severità.

Non scatta mai su un promemoria di un'anomalia già nota: una coda che oscilla di poco (es. 15→18 messaggi) non genera questa mail.

Oggetto con tag dedicato per le code. Se una qualunque coda raggiunge quella stessa soglia di 200 messaggi, l'oggetto della mail di riepilogo guadagna il tag **"ATTENZIONE CODE"**, visibile senza aprire il messaggio.

6. Programmazione automatica

```
.\Install-ExchangeHealthCheckTask.ps1 -IntervalMinutes 15 -UserName 'DOMINIO\account-servizio'
```

Registra un'attività pianificata che parte all'avvio del server e si ripete ogni N minuti. Un mutex globale impedisce esecuzioni sovrapposte se un giro dura più dell'intervallo.

Per rimuoverla: `.\Install-ExchangeHealthCheckTask.ps1 -Unregister`

7. Domande frequenti

Come faccio a sapere che versione dello script ho in esecuzione? È la prima riga di log di ogni esecuzione: `Invoke-ExchangeHealthCheck.ps1 - versione X.Y.Z`, subito dopo la riga di separatore. Utile soprattutto quando aggiorni lo script su una macchina diversa da quella su cui lo sviluppi: se il problema che stai vedendo dovrebbe essere già risolto ma persiste identico, la prima cosa da controllare è che la macchina di esecuzione abbia davvero l'ultima versione — il numero è anche nel file `VERSION` alla radice del repository, da confrontare senza dover aprire l'intero script.

"WinRM cannot find the computer" su un server acceso e funzionante Quasi sempre è risoluzione nome, non un server giù. Verifica con l'FQDN: `Invoke-Command -ComputerName server.tuodominio.local {`

`$env:COMPUTERNAME }`. Lo script usa già l'FQDN di default (`Servers.UseFqdnForRemoting`).

Lo script gira su un server e proprio quel server risulta irraggiungibile Comportamento noto e già gestito: il server locale non passa da WinRM, viene interrogato direttamente. Se accade, aggiorna alla versione più recente.

"Nessun DAG rilevato" Normale su un server standalone senza DAG: i check DAG/replica vengono saltati, il resto continua.

Il controllo dei nodi cluster non compare mai, nemmeno come errore Serve il cmdlet `Get-ClusterNode`, che richiede lo strumento RSAT "Failover Clustering Tools" (`RSAT-Clustering-PowerShell`) installato sulla macchina da cui esegui lo script — non basta avere Exchange. Se manca, dalla versione più recente ricevi un finding `Warning` esplicito che te lo dice; se invece non vedi nulla, aggiorna alla versione più recente.

"object ... could not be found on " Prova `"Organization": { "ViewEntireForest": false }`. Se persiste, imposta `PreferredDomainController` con l'FQDN di un DC della stessa foresta dei server Exchange (non basta la stessa foresta: serve anche che il DC replichi il Configuration NC). Nota anche che alcuni cmdlet (es. lo stato delle copie database) rispondono lo stesso messaggio quando il server semplicemente non ospita copie, o quando l'oggetto è fuori dallo scope RBAC: non è sempre un problema di DC.

Back pressure segnalata come "Low" Low è lo stato sano. Se lo vedi ancora segnalato come anomalia, aggiorna lo script: era un bug risolto (i campi venivano letti invertiti).

ForwardSyncDaemon o ProvisioningRps sempre Inactive Normale: sono componenti usati solo dal datacenter Microsoft, su on-premises restano Inactive per progetto. Sono esclusi di default.

Due certificati con lo stesso nome, uno scaduto I connector Exchange identificano i certificati per Subject+Issuer, non per thumbprint: con due certificati identici in quei campi, Exchange può agganciare quello sbagliato. Lo script segnala automaticamente i duplicati con questa caratteristica.

"Mailbox unavailable / Unable to relay / Recipient not in accepted domain" Il destinatario della mail non è in un dominio che Exchange considera suo: è relay, negato per l'invio anonimo. Usa una casella interna, oppure autentica l'invio (`Mail.UseDefaultCredentials` o `Mail.CredentialFile`), oppure crea un receive connector dedicato limitato per IP.

"Cannot convert value ... DisplayHint ... to type System.DateTime" Bug risolto: sotto Windows PowerShell 5.1, una data salvata in un certo modo nello stato degli alert poteva diventare illeggibile al giro successivo. Aggiorna alla versione più recente dello script. Se il file di stato è già danneggiato, cancellalo (si ricostruisce da solo, e gli alert già noti verranno rinotificati una volta come "nuovi"):

```
Remove-Item .\State\alert-state.json
```

Il giro sembra bloccato per minuti, senza errori Guarda l'ultima riga di log con attenzione: la durata di un check si stampa solo quando **finisce**, quindi il nome che vedi fermo da minuti è quasi sempre quello **appena completato**, non quello bloccato — il vero responsabile è il check *successivo*, ancora in corso. I cmdlet `Get-ExchangeCertificate` e `Get-HealthReport` non hanno un timeout proprio: se il servizio a cui fanno RPC su un server è inceppato, restano appesi anche per diversi minuti (il timeout RPC di default di Windows è 300 secondi). Dalla versione più recente, questi due controlli hanno un limite configurabile (default 30 secondi — `Thresholds.CertificateCheckTimeoutSeconds` e `Thresholds.ManagedAvailabilityTimeoutSeconds`): oltre quel limite il check viene abbandonato e segnato come non riuscito, invece di bloccare il resto del giro sugli altri server.

Cos'è la "replay queue" (e la "copy queue")? Sono la misura di quanto una copia passiva di un database è indietro rispetto a quella attiva. La **copy queue** conta i log di transazione già copiati sul disco della copia passiva ma non ancora presenti nel suo storage vero e proprio; la **replay queue** conta quelli già copiati e riprodotti (replayed) nel database, cioè effettivamente applicati. In un DAG sano entrambe oscillano vicino a zero — un valore che sale e resta alto indica che quella copia non sta più tenendo il passo (rete lenta, disco saturo, copia in

seeding). Le soglie sono `Thresholds.CopyQueueWarning/CopyQueueCritical` e `Thresholds.ReplayQueueWarning/ReplayQueueCritical`; non si applicano alla copia attiva (`Mounted`), che per definizione non ha nulla da recuperare.

Una copia mostrata come "Healthy" nel riepilogo per database, ma con un finding Critical su ReplayQueue

Non è una contraddizione: lo `Status` di Exchange (`Healthy`) descrive solo che il meccanismo di copia funziona correttamente, non che la copia sia allineata — può restare `Healthy` anche con migliaia di log di transazione ancora da riprodurre (replay) sul database. Bug risolto: nelle versioni precedenti la severità mostrata nei riepiloghi (per server e per database) guardava solo questo `Status` testuale, ignorando `CopyQueueLength/ReplayQueueLength` — una copia poteva quindi apparire sana nello specchietto pur avendo già un finding `ReplayQueue Critical` tra le anomalie. Dalla versione più recente la severità di riga considera anche le code, e il dettaglio mostra sempre `RQ:<valore>` per capire subito la causa senza dover interrogare `Get-MailboxDatabaseCopyStatus` a mano.

Content index in stato "NotApplicable" Non è un guasto: è uno stato normale, tipico di una copia database **ritardata** (lagged copy — ha un `ReplayLagTime` configurato) che Exchange non indicizza di proposito, perché non è pensata per servire ricerche live. Aggiorna alla versione più recente: nelle versioni precedenti veniva segnalato per errore come `Critical`.

"Check non eseguibile — Cannot bind parameter 'ConnectionUri' ... hostname could not be parsed" Non è un problema di rete o DNS, anche se l'errore lo sembra: è la sessione Exchange verso cui lo script sta lavorando che è caduta, e il tentativo di riconnessione automatica interno di Exchange fallisce con questo errore fuorviante che non c'entra nulla con l'operazione richiesta. Dalla versione più recente, quando lo script rileva questo sintomo pulisce la sessione caduta, prova a riconnettersi e ripete il check una volta sola — sia che la sessione l'avesse aperta lo script stesso (`Organization.ConnectTo` configurato), sia che tu stia eseguendo lo script dentro una Exchange Management Shell o una sessione aperta a mano con `Connect-ExchangeServer/RemoteExchange.ps1` già aperta prima di lanciarlo. Se vedi ancora l'errore dopo l'aggiornamento, il finding `Unknown` te lo dice esplicitamente: in quel caso la sessione era genuinamente esterna e senza `Organization.ConnectTo` configurato né lo snap-in locale registrato lo script non ha modo di ricrearla da solo — riapri la sessione Exchange (nuova Exchange Management Shell o `Connect-ExchangeServer`) e rilancia lo script.

"Access is denied" su Get-Queue comparso su tutti i server, dopo aver visto l'errore sopra Bug risolto, causato proprio dal fix precedente: in un ambiente senza `Organization.ConnectTo` (sessione Exchange già aperta da una Exchange Management Shell o da `Connect-ExchangeServer`), i controlli `Certificate` e `ManagedAvailability` girano in un processo separato con un timeout duro (protezione contro cmdlet Exchange che restano appesi) che **non eredita** quella sessione esterna — quindi falliva sempre, non per una sessione davvero caduta. Il recupero pensato per le sessioni cadute veniva innescato comunque, e ripuliva per errore anche il modulo Exchange valido della sessione esterna, condiviso con tutti gli altri check. Aggiorna alla versione più recente: quei due controlli ora, in questa configurazione, girano diretti senza passare dal processo separato (perdendo solo la protezione da timeout duro per loro due, non il resto).

Il totale messaggi in coda di un server segnala "Warning" ma nessuna coda singola sembra alta Bug risolto: il totale sommava tutte le code del server (anche tante code piccole, ciascuna con pochi messaggi) e lo confrontava con la stessa soglia usata per una singola coda in stallo. Dalla versione più recente il totale ha una soglia propria, più alta (`Thresholds.QueueTotalWarning/QueueTotalCritical`, default 300/1000), pensata per un backlog reale sull'intero server; la soglia per singola coda (`QueueWarning/QueueCritical`) resta quella di prima e continua a segnalare subito una coda davvero bloccata.

L'health set "Search" è Unhealthy su un server, ma non ha nessuna copia di database Non è un errore: se un server non ospita nessuna copia di database (né attiva né passiva), non c'è alcun indice di ricerca locale da mantenere, quindi l'health set non riflette alcun impatto reale. Lo script riconosce automaticamente questo caso e non genera più un alert `Critical/Warning` per `Search` in questa situazione — resta comunque visibile come finding `Info` tra i dettagli, per trasparenza, ma senza scatenare una notifica. Configurabile in `Ignore.HealthSetsWithoutDatabaseCopy` (default: solo `Search`). Gli altri health set di tipo proxy/protocollo

(ActiveSync, Imap, OWA.Calendar.Proxy, Outlook.Proxy/MapiHttp.Proxy) **non** vengono soppressi allo stesso modo: sono funzioni di front-end che un server esercita per qualunque utente dell'organizzazione, non solo per le cassette che ospita localmente.

Vedo una ReplayQueue/CopyQueue in Warning nel report, ma non ho ricevuto una mail di alert

Comportamento voluto, non un bug: `ReplayQueue/CopyQueue` oscillano normalmente in Warning durante una replica pesante (dopo un riavvio, un failover, un picco di scrittura) e spesso rientrano da sole in pochi minuti — non un incidente. Lo script ora aspetta che restino elevate **ininterrottamente per almeno 2 ore** (default, configurabile in `Alerting.SustainedDurationMinutes`) prima di inviare un alert per queste due categorie specifiche (`Alerting.SustainedCategories`); se rientrano prima, non arriva nessuna mail, ma restano comunque visibili nel report/console/log di ogni giro nel frattempo. Tutte le altre categorie (certificati, componenti, ActiveSync, ecc.) continuano a notificare subito come prima — questo comportamento è specifico solo per le code di replica del DAG. Per un test immediato senza aspettare 2 ore, `-ForceMail/-TestMail` bypassano sempre l'attesa.

Vedo una nuova severità "Low Issue" (colore ambra) — cos'è? È un livello sotto Warning, pensato per anomalie note come poco urgenti in questo ambiente: gli health set `ActiveSync`, `OWACalendar.Proxy` e `Imap` (Managed Availability) e l'intera categoria `Certificate` vengono sempre mostrati come Low Issue invece di Warning/Critical (configurabile in `LowIssueKeys`), e un disco con meno del 7% di spazio libero — anche se in GB assoluti è ancora tanto, tipico dei volumi molto grandi — genera un Low Issue invece di restare invisibile. Un Low Issue **non genera mai una mail di alert** (resta sotto la soglia minima di notifica di default), ma è sempre visibile nel report/console, nella sezione "Stato per categoria" e in una tabella dedicata "Anomalie a basso impatto (Low Issue)" della mail — è un "tienilo d'occhio", non un "non fare nulla".

Attenzione: siccome `Certificate` è declassata **per intero**, anche un certificato già scaduto oggi genera solo un Low Issue, non un alert via mail — se preferisci un comportamento diverso per quel caso specifico, richiedi una modifica a `LowIssueKeys`.

8. Sicurezza e privacy dei dati

- Lo script non modifica nulla in Exchange: solo cmdlet `Get-*` e `Test-*`.
- La configurazione reale, i log, i report CSV e lo stato degli alert contengono nomi di server, database e indirizzi del tuo ambiente: non vanno mai condivisi o versionati (già esclusi di default in `.gitignore`).
- Le credenziali SMTP, se usate, sono salvate con `Export-Clixml` (cifatura DPAPI legata all'account che le genera): vanno create con lo stesso account che eseguirà lo scheduled task.

Documento generato insieme allo sviluppo dello script. Va aggiornato ad ogni modifica funzionale rilevante — vedi `HANDOFF.md` nel repository per il dettaglio tecnico completo e la cronologia dei fix.